Prototype-Initialized Adaptation of ViT-B/16 for 10,000-Class Fine-Grained Classification under a Fixed Compute Budget

Member 1, Member 2, Member 3 & Member 4
CS604 Team 06, Singapore Management University
{email addresses}

Abstract

We adapt ViT-B/16 to a fine-grained dataset with 10,000 classes and only 45 training images per class, with 2 hours on 48 GB of GPU memory per training run. We compare seven adaptation methods under the same settings, from a linear probe to full fine-tuning, BitFit, Adapter, AdaptFormer and LoRA. The choice that mattered most was how to start the new 10,000-class head. The usual start, a linear probe trained on frozen features of the training images, made all three methods we tested worse; starting each class from its mean feature (a prototype) gave 2.1–3.1 points more. We also find that where the trainable parameters are matters more than how many there are: LoRA on 1.375% of the backbone matches fine-tuning the last four blocks (33%). Our final full fine-tuning model reaches 76.22% top-1 on the CodaBench validation set and 76.09% on the test set (reference: 70.8%) in 1.03 GPU-hours, and our final LoRA model reaches 71.90% and 71.67% (reference LoRA: 68.3%) with no extra inference cost. We also report the ideas that failed, and show after the deadline that larger input images add another 4.4 points.

1 Introduction

Pretrained Vision Transformers (Dosovitskiy et al., 2021; Steiner et al., 2022) are usually adapted to a new task instead of being trained from scratch, either by fine-tuning all weights or by training a small number of added parameters (Houlsby et al., 2019; Hu et al., 2022; Chen et al., 2022). In this project we adapt ViT-B/16 to a fine-grained dataset with 10,000 classes whose names are hidden. There are 45 training images per class (450,000 in total); the labels of the validation set (50,000 images) and the test set (100,000 images) are hidden and scored on CodaBench. Each training run may use 2 hours on 48 GB of GPU memory, which is 1.2 hours on our 80 GB A100.

Why this is hard. As Fig. 1 shows, one class can look very different from image to image, while different classes in a fine-grained dataset can look alike, and each class has only 45 images. A model

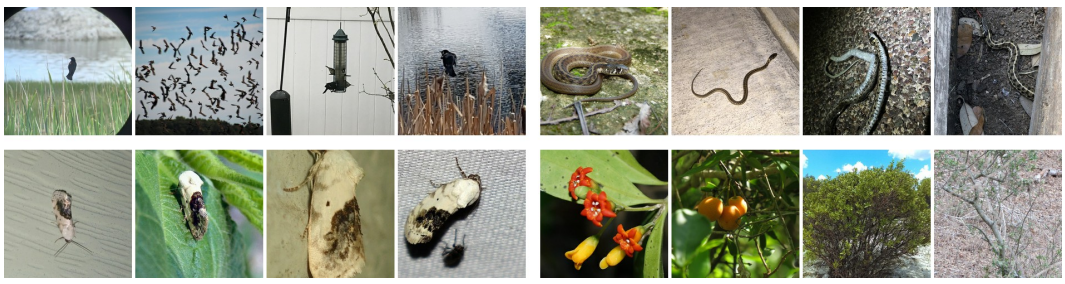


Figure 1: **Training images of four of the 10,000 classes.** Each group of four is one class (a bird, a snake, a moth and a plant): one bird or a whole flock, a flower or a whole shrub. Each class has only 45 such images.

can easily memorize such a small set, so a low training loss tells us little; every choice has to be judged on held-out images. The new classifier head is also large: a $10{,}000 \times 768$ layer has 7.69M parameters, 9% of the 85.8M-parameter backbone, and each of its rows is trained on only 43–45 images. Finally, the budget is tight: one epoch of full fine-tuning takes about 7 minutes, so only about eight epochs fit into one run.

Our objectives. A high leaderboard score was not our only goal. We wanted to answer three questions. (1) What limits accuracy when there are many classes but few images per class, and how can we fix it? (2) If we compare adaptation methods fairly, with one protocol and one budget, where do trainable parameters help most, and what does each method cost in memory and latency? (3) How should a fixed budget be split between epochs, training data and test-time computation? We answered them with 26 runs, each changing one setting, and chose models on a local holdout split that matches the CodaBench validation score within 0.3 points. Our contributions are:

- **A better start for the head.** The linear-probe (LP) head used by LP-FT (Kumar et al., 2022), trained on frozen features of the training images, hurts in our task. Starting each class from its normalized mean feature (a prototype) instead adds 2.1–3.1 points for full fine-tuning, LoRA and AdaptFormer. A random-head control shows that the harm comes from fitting the head on the training images (Section 4.4).
- **A fair comparison of seven adaptation methods.** Small changes in every block give much more accuracy per parameter than fine-tuning the top blocks: LoRA on 1.375% of the backbone matches the last four blocks (33%). Training memory depends on how far back the gradients go, not on how many parameters are trained (Section 4.5).
- **Final results with their full cost.** Our final models reach 76.22% (full fine-tuning, +5.4 over the reference) and 71.90% (LoRA, +3.6 over the reference LoRA) on the validation set, and 76.09% and 71.67% on the test set, within the budget (Section 4.3).
- **The process, including failures.** We report the study round by round, with the ideas that failed (Section 4.2), and find after the deadline that larger input images add 4.4 points (Section 4.7).

2 Related Work

Pretrained ViTs and fine-tuning. ViT (Dosovitskiy et al., 2021) applies a Transformer (Vaswani et al., 2017) to 16×16 image patches. Pretraining on ImageNet-21k (Deng et al., 2009) with strong augmentation and regularization (AugReg, Steiner et al., 2022) gives backbones that transfer well, and models that are better on ImageNet also transfer better (Kornblith et al., 2019). Common fine-tuning recipes use AdamW (Loshchilov & Hutter, 2019), warmup with cosine decay (Loshchilov & Hutter, 2017), label smoothing (Szegedy et al., 2016), stochastic depth (Huang et al., 2016) and layer-wise learning-rate decay (Bao et al., 2022); our full fine-tuning recipe uses all of them.

Parameter-efficient fine-tuning (PEFT). PEFT methods train only a small part of the model. The oldest option is to train only the top layers, whose features are the most task-specific (Yosinski et al., 2014). BitFit (Ben Zaken et al., 2022) trains only the bias terms. Adapters (Houlsby et al., 2019) add a small bottleneck MLP to every block, and AdaptFormer (Chen et al., 2022) puts a scaled bottleneck next to the MLP instead of after it. LoRA (Hu et al., 2022) learns a low-rank update $\Delta W = BA$ of frozen weight matrices, which can be merged into the weights after training. VPT (Jia et al., 2022) and prefix tuning (Li & Liang, 2021) add learnable tokens instead. Table 1 lists the pros and cons we expected from the literature. No method is best on every count: they trade accuracy against training memory, stored size and inference latency, so we measure all of these.

Starting the classifier head. When a pretrained model is moved to new classes, its head is replaced; ViT starts the new head at zero (Dosovitskiy et al., 2021). LP-FT (Kumar et al., 2022) argues that a random head distorts the pretrained features during fine-tuning, which hurts accuracy on new domains. It therefore first trains a linear probe on frozen features and then fine-tunes everything. Prototypical networks (Snell et al., 2017) represent each class by the mean feature of its examples, and weight imprinting (Qi et al., 2018) copies normalized class means directly into the weights of a cosine classifier, to add new classes from a few examples. We use imprinting as the starting head for

Table 1: **Adaptation methods and head starts: pros and cons**, as expected from the literature. Our measured numbers are in Section 4.

Approach	Pros	Cons
Full fine-tuning	most accurate with enough data	most memory; a full model copy per task
Linear probe	cheapest; no backbone gradients	the features never adapt
Top-K blocks	little memory (short backward pass)	the lower blocks never adapt
BitFit	tiny; no extra latency	little capacity
Adapter	small; flexible	extra layers slow down inference
AdaptFormer	small; keeps the MLP output	cannot be merged; slower inference
LoRA	small; merged after training, no extra latency	gradients still pass through every block
VPT, prefix tuning	very small	longer input, so slower; needs tuning of the token count
Random or zero head	simple	large, noisy gradients at the start
LP-FT head	protects the pretrained features	fitted on the training images (hurts here, Section 4.4)
Imprinted prototypes	no training; uses the class structure	only as good as the frozen features

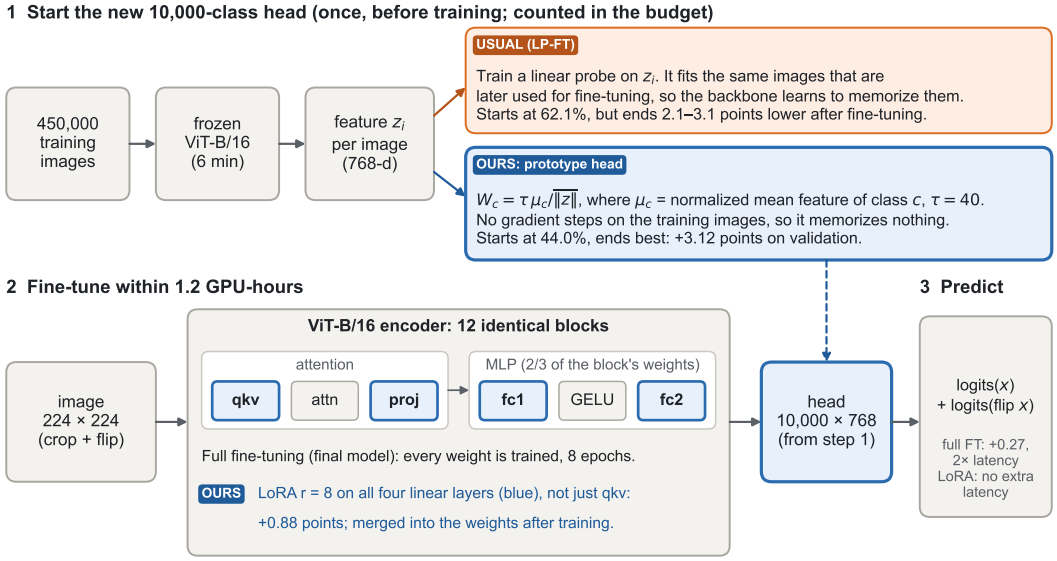


Figure 2: **Our pipeline, with our changes in blue.** (1) We start the new head from class prototypes of the frozen backbone, not from the usual LP-FT linear probe (orange), which is fitted on the same training images. (2) We fine-tune all weights, or LoRA on all four linear layers (the reference: qkv only). (3) Full fine-tuning adds the logits of each image and its mirror image. Gains are from Section 4.

full fine-tuning and PEFT, and compare it with LP-FT in a setting with many classes (10,000), few images per class (45) and a short budget.

3 Method

Figure 2 shows our pipeline. Let f_θ map an image x to its final `[CLS]` feature $z = f_\theta(x) \in \mathbb{R}^{768}$, and let $h(z) = Wz + b$ with $W \in \mathbb{R}^{10000 \times 768}$ be the new head.

What we changed, and why. We started from the recipes in the project description and changed five things, each after a test on the holdout (Section 4.2):

1. **A prototype head instead of a linear-probe head** (Section 3.2). Our first runs used LP-FT. When LoRA's training loss went *up* in its first epoch, we traced the problem to the probe, which had been fitted on the same images. A prototype head is not trained on them at all. This is our main change: $+2.1$ to $+3.1$ points.
2. **LoRA on all four linear layers, not only qkv**, because the MLP holds two thirds of each block's weights: $+0.88$ points with 1.375% of the backbone.
3. **A different schedule for each final model**: 8 epochs for full fine-tuning, which keeps improving, and 3 for LoRA, which gets worse with more epochs (Section 4.6).
4. **A budget clock that counts every GPU second**, including the feature extraction for the head, and shortens the schedule if a run would go over the budget (Section 3.1).
5. **Flip test-time augmentation** for the full fine-tuning model: $+0.27$ points for twice the latency.

3.1 Backbone and budget

Backbone. We use the timm (Wightman, 2019) checkpoint `vit_base_patch16_224.augreg2_in21k_ft_in1k`, the backbone that the project description recommends: ViT-B/16 pretrained on ImageNet-21k with AugReg and fine-tuned on ImageNet-1k. It has 85,798,656 parameters; its 1,000-class head is replaced by a new 10,000-class head (7,690,000 parameters). We keep it in every experiment: after the deadline we checked two ImageNet-21k-only ViT-B/16 models that the project description also lists, and a linear probe on their frozen features is worse on our holdout (61.05% and 51.07% *vs*. 61.94%).

Compute budget. The project allows $2\,\text{h} \times 48\,\text{GB} = 96\,\text{GB·h}$ per training run ($\pm 10\%$), which is 1.2 hours on our 80 GB A100. Our clock counts all GPU work for training (head initialization with feature extraction, all optimizer steps and the holdout check after every epoch), but not unpacking the data, saving checkpoints or the final predictions. The code times training steps 20–120; if the planned epochs would use more than 97% of the budget, it shortens the cosine schedule, and it stops training at $1.05\times$ the budget. Every run in our results finished within 1.06 GPU-hours; the only longer run is a post-deadline test that is not part of our results (Section 4.7).

3.2 Starting the 10,000-class head

The head is the largest new part of the model, and each of its rows is trained on at most 45 images. A random head gives almost uniform logits (initial loss $\approx \ln 10000 = 9.2$), so early in training the backbone gets large, noisy gradients from it. We compare three ways to start the head.

Random. timm's default initialization of a new linear layer.

Prototype head (ours). With the frozen pretrained backbone, we extract the features z_i of all training images once. We then copy the L2-normalized class means into the head (Snell et al., 2017; Qi et al., 2018):

$$\mu_c = \frac{\sum_{i:\, y_i = c} \hat{z}_i}{\left\| \sum_{i:\, y_i = c} \hat{z}_i \right\|}, \quad W_c = \frac{\tau}{\overline{\|z\|}}\, \mu_c, \quad b_c = 0, \tag{1}$$

where $\hat{z}_i = z_i / \|z_i\|$ and $\overline{\|z\|}$ is the mean feature norm, so each logit is about $\tau \cos(z, \mu_c)$. We pick the temperature τ from $\{5, 10, 15, 20, 30, 40, 60, 80\}$ by the cross-entropy on 10,000 training features, which gives $\tau = 40$. Without any training, this head already reaches 44.0% holdout accuracy. Feature extraction takes 6–7 minutes on an A100, and we count this time in every run's GPU-hours. The head knows the class structure from the start, but it has never been fitted to single training images, so it has nothing to memorize.

Linear-probe (LP) head. Starting from the prototype head, we train (W, b) on the cached frozen features for 10 epochs (AdamW, learning rate 10^{-3}, weight decay 10^{-4}, batch size 4,096, cosine decay, label smoothing 0.1). This reaches 62.1% holdout accuracy in a few seconds and is the first stage of LP-FT (Kumar et al., 2022). It was our first design, chosen so that a random head would not distort the features.

3.3 Adaptation methods

Full fine-tuning. All backbone and head weights are trained with AdamW (Loshchilov & Hutter, 2019) at learning rate 10^{-4} and weight decay 0.05 (none on biases, normalization parameters, the class token and position embeddings). We use layer-wise learning-rate decay of 0.75 (Bao et al., 2022), drop path 0.1 (Huang et al., 2016), a linear warmup (one epoch for runs of 6 or more epochs, 0.3 epochs for 3-epoch runs) and a cosine decay to zero (Loshchilov & Hutter, 2017). Other settings are batch size 256, bf16 mixed precision, gradient clipping at norm 1 and label smoothing 0.1 (Szegedy et al., 2016). Training images get RandomResizedCrop (scale 0.25–1) and a horizontal flip; for evaluation, images are resized to 248 pixels and center-cropped to 224. At test time, the final full fine-tuning model adds the logits of each image and of its horizontal flip (Krizhevsky et al., 2012).

LoRA. For a frozen linear layer $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, LoRA (Hu et al., 2022) computes

$$y = W_0 x + \tfrac{\alpha}{r} BAx, \tag{2}$$

where $A \in \mathbb{R}^{r \times d_{\text{in}}}$ and $B \in \mathbb{R}^{d_{\text{out}} \times r}$ are trained and B starts at zero, so training starts exactly from the pretrained model. We use the PEFT library (Mangrulkar et al., 2022) with $r = 8$ and $\alpha = 16$. timm joins the query, key and value projections into one `qkv` layer, so LoRA on `qkv` (294,912 parameters, 0.344% of the backbone) adapts all three. Our final LoRA model applies LoRA to all four linear layers of every block (`qkv`, `proj`, `fc1`, `fc2`), for 1,179,648 parameters (1.375%). The LoRA weights and the head use learning rate 10^{-3} and weight decay 10^{-4}, with no LoRA dropout. After training, $\frac{\alpha}{r} BA$ is added into W_0, so inference uses exactly the base model.

Other methods. AdaptFormer (Chen et al., 2022) adds a bottleneck branch next to the MLP of each block,

$$x_{l+1} = x' + \text{MLP}(\text{LN}(x')) + s \cdot W_{\text{up}} \, \text{ReLU}(W_{\text{down}} x'), \tag{3}$$

where x' is the output of the attention sub-block; we use width 64, scale $s = 0.1$, dropout 0.1 and a zero-initialized W_{up}. The serial Adapter (Houlsby et al., 2019) applies the same bottleneck to the MLP output, before the residual addition and without a scale. Both have 1,189,632 parameters (1.387%) and cannot be merged into the backbone. BitFit (Ben Zaken et al., 2022) trains only the bias vectors of all linear and normalization layers (102,912 parameters, 0.120%). Partial fine-tuning trains the last K blocks and the final normalization layer; one block has 7,087,872 parameters (8.26% of the backbone). Table 8 in the appendix shows how each count is computed. In every run except zero-shot the head is also trained; since it alone has 7.69M parameters, we report trainable parameters without it, as in the project's 0.34% LoRA reference.

4 Experiments

4.1 Setup

Data and protocol. Validation and test labels are hidden, so we hold out 2 images per class from the training set (20,000 images, fixed seed). We use this holdout to choose models and run ablations, and train on the other 43 images per class. The final full fine-tuning model is then retrained on all 45 images per class; the final LoRA model keeps the holdout split and is trained on 43. Validation and test images are used only to make submission files. CodaBench reports top-1 accuracy; the validation set is balanced (5 images per class), so this is also the mean per-class accuracy. Our holdout is close to the server score: 72.31% *vs.* 72.57% for full fine-tuning and 71.98% *vs.* 71.90% for LoRA. At about 72% accuracy on 20,000 images, the standard error is $\sqrt{0.72 \cdot 0.28/20{,}000} \approx 0.32$ points, so we treat differences below about 0.3 points as noise. All runs use one seed.

Metrics. Latency is the mean time of one forward pass at batch size 1 on an A100 (bf16, 100 passes after 10 warm-up passes); for the same model it varies by up to about 0.5 ms between runs. Throughput is measured at batch size 256, GPU-hours come from the clock in Section 3.1, and memory is the peak allocated by PyTorch during training. The appendix gives training-log excerpts, all runs, the hyperparameters and the hardware (Appendices A to C and E).

Table 2: **How we iterated**, one question per round (holdout top-1 unless marked val; FT: fine-tuning). *After the deadline; no ranked submission uses it.

Date	Question → experiment	Finding
Sep 28	Is our pipeline right? → reproduce the 3 references	all within 0.3 points
Sep 28	Do longer runs help? → full FT for 8 epochs, LoRA for 10	full +1.4, LoRA +0.1; LoRA's loss goes up in epoch 1
Oct 18	Is LoRA's learning rate too high? → halve it	**failed**: −0.12 (Table 9)
Oct 18	Does LoRA need more capacity? → adapt the MLP too	+0.88 (Table 9)
Oct 19	Is the LP head the problem? → LP *vs.* random *vs.* prototype	prototype +2.5; random ties it (Table 4)
Oct 19	Final recipe → prototype head; full FT 8 ep.; LoRA 3 *vs.* 6 ep.	val 76.22, 71.90; LoRA 6 ep. **failed**: −0.46
Nov 2	Which method is best per parameter? → seven methods, one protocol	small changes in every block (Table 5)
Nov 5*	What limits us now? → in21k-only backbones; larger images	backbone: no gain; image size: +4.4 (Table 6)

Table 3: **Main results.** Holdout: 2 images per class held out from the training set; Val/Test: CodaBench validation/test (Phase 2). Params: trainable backbone parameters (share of the 85.8M backbone); the 10,000-class head (7.69M) is also trained in every row except zero-shot. Latency (ms, batch 1), throughput, GPU-hours and peak memory: one A100; LoRA after merging. †All 45 images per class; other rows (incl. the final LoRA model) use 43 and no TTA. ‡Flip TTA (two forward passes; 7.35 ms without).

Method	Head	Ep.	Params	Holdout	Val	Test	ms	img/s	GPU-h	GB
Zero-shot	prototype	0	0	43.95	–	–	7.55	3,832	0.11	3.9
Linear probe	LP	–	0	62.07	–	–	–	–	≈0.11	–
Full fine-tuning	LP	8	85.8M (100%)	72.31	72.57	–	7.35	3,864	1.03	18.9
Full fine-tuning (final)†	prototype	8	85.8M (100%)	–	**76.22**	**76.09**	14.95‡	1,919‡	1.03	18.9
LoRA, qkv only	LP	3	0.29M (0.344%)	68.57	–	–	7.22	3,817	0.40	13.5
AdaptFormer	LP	3	1.19M (1.387%)	69.05	–	–	9.89	3,461	0.39	12.8
LoRA, all linear (final)	prototype	3	1.18M (1.375%)	**71.98**	**71.90**	**71.67**	7.61	3,869	0.54	19.9

4.2 How we iterated

We ran the study in rounds; each round asked one question and changed one setting (Table 2). The failed ideas were useful too: when LoRA's training loss went up in its first epoch, we first blamed the learning rate, but halving it changed nothing, so we looked at the head next, which gave the largest gain of the project (Section 4.4).

Reproducing the reference baselines. Before changing anything, we checked our pipeline against the three reference results in the project description. Short 3-epoch runs with the LP head match all three within 0.3 points on our holdout (43.95% *vs.* 43.7% for the zero-shot prototype head, 70.88% *vs.* 70.8% for full fine-tuning, 68.57% *vs.* 68.3% for LoRA on qkv), so our pipeline and the way we count trainable parameters agree with the reference.

4.3 Main results

Table 3 gives all four required metrics for the key models. Our final full fine-tuning model reaches **76.22%** on the CodaBench validation set, 5.4 points above the reference. Changing one setting at a time (Fig. 6 and Table 10 in the appendix), replacing the LP head with the prototype head adds **3.12 points**, going from 3 to 8 epochs adds 2.21, flip TTA adds 0.27 (it changes 4.6% of the predictions and doubles latency), and training on all 45 images per class adds 0.26 (close to the noise, but in the expected direction). Our final LoRA model reaches **71.90%** (+3.6 over the reference LoRA); it trains 1.375% of the backbone, and after merging it has the same latency as the base model. The two final models were not trained the same way: the LoRA model keeps the holdout split and uses no TTA. Under the same settings (3 epochs, prototype head, holdout), LoRA reaches 98% of the full fine-tuning accuracy (71.98% *vs.* 73.38%). On the hidden test set (Phase 2), the two final models

Table 4: **Head initialization** (top-1, %); in each column only the head changes. Head alone: before training. Full FT 8 ep: all 45 images per class, flip TTA. *RTX PRO 6000; the A100 rerun gives 71.98.

Head init	Head alone	LoRA, all linear 3 ep, holdout	AdaptFormer 3 ep, holdout	Full FT 3 ep, holdout	Full FT 8 ep, val
Random	chance	71.95*	–	–	–
Prototype (ours)	44.0	**72.00***	**71.17**	**73.38**	**76.22**
Linear probe (LP)	62.1	69.45	69.05	70.88	73.10

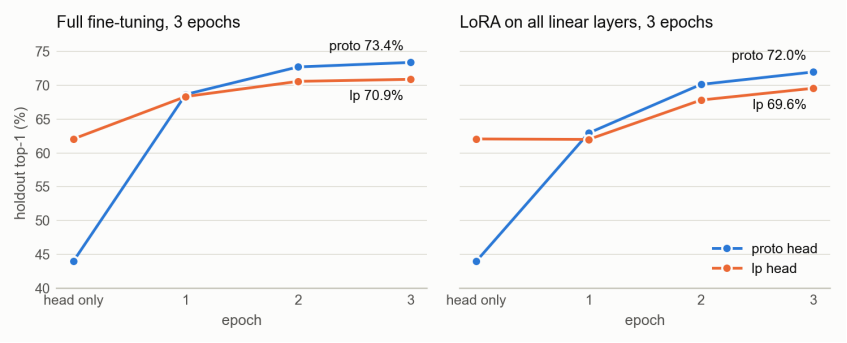


Figure 3: **Holdout accuracy during training for the LP and prototype heads** (3 epochs; left: full fine-tuning, right: LoRA on all linear layers). "head only" is the accuracy of the starting head before any training. The LP head starts 18 points higher but ends about 2.5 points lower. Points are the holdout checks after each epoch; for LoRA with the LP head, the final check after merging gives 69.45 (Table 4) instead of 69.56.

reach **76.09%** and **71.67%**, within 0.25 points of their validation scores. Both models were fixed before the test set was released, and each used one of our two Phase 2 submissions.

4.4 Starting the head

Table 4 compares the three head starts in four settings. In all four, the LP head is clearly the worst, 2.1–3.1 points below the prototype head, although it starts 18 points higher (Fig. 3). For LoRA, the random and prototype heads end up level (71.95% *vs*. 72.00%), but the prototype head learns faster: it is 5 points ahead after the first epoch (63.3% *vs*. 58.4%).

Why does the LP head hurt? The probe was trained on *the same images that are then used for fine-tuning*, so it already fits them well. Over the first 100 steps of the LoRA runs, the mean training loss is 2.29 with the LP head, against 4.39 with the prototype head and 9.28 with a random head. So the backbone gets only a small training signal, mostly from the few images the probe gets wrong. When the features start to change, the trained head no longer matches them: in the LoRA run with the LP head, the training loss goes up from 2.22 to 2.69 during the first epoch, before any image is seen twice. Figure 4 shows the same for full fine-tuning: with the same settings, the LP model reaches a lower training loss but a much lower validation accuracy, so it memorizes more and generalizes less. The random head is our control: it knows nothing about the classes, yet it does as well as the prototype head. So the prototype head is not special; what hurts is a head that was trained on the training images. LP-FT (Kumar et al., 2022) uses the probe so that a random head does not distort the pretrained features, mainly to keep accuracy on new domains. In our task the features have to change, and a random head did no harm.

4.5 Comparing adaptation methods

To compare the methods fairly, we ran all seven with the same protocol: the prototype head, 3 epochs, the same holdout split and the same GPU (Table 5 and Fig. 5). All efficient methods use learning rate 10^{-3}; we did not tune it for each method.

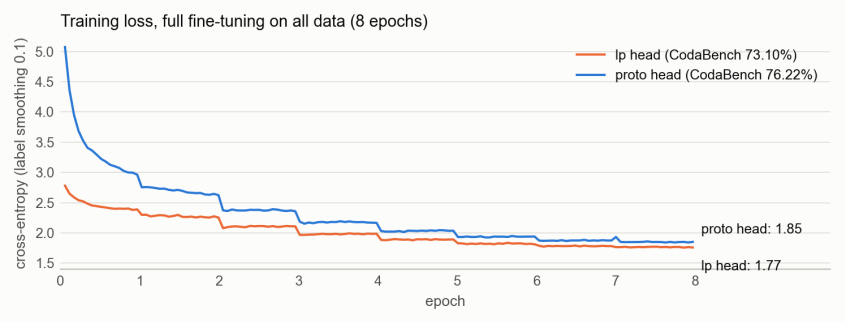


Figure 4: **Training loss of full fine-tuning on all data (8 epochs) with the LP and prototype heads**; nothing else differs. The prototype model has a *higher* training loss (1.85 *vs*. 1.77 in the last epoch) but scores 3.12 points higher on the validation set. The loss drops at each epoch boundary and stays flat within an epoch, a sign that the model memorizes the 45 images per class.

Table 5: **Seven adaptation methods under one protocol** (prototype head, 3 epochs, holdout top-1, %). Params: trainable backbone share; img/s: training throughput; GB: peak training memory; ms: batch-1 latency. *Peak during feature extraction; training used less. †Full fine-tuning recipe; $K = 1, 2$ use learning rate $3{\times}10^{-4}$ without layer decay or drop path.

Method	Params	Holdout	img/s	GB	ms
Linear probe	0	61.94	–	3.2	6.96
Partial, $K = 1$	8.26%	67.97	1,706	3.8*	7.92
Partial, $K = 2$	16.5%	70.99	1,655	4.1	7.41
Partial, $K = 4$†	33.0%	71.88	1,611	7.0	7.41
Partial, $K = 6$†	49.6%	72.89	1,650	10.0	7.42
Full fine-tuning	100%	**73.38**	1,139	18.9	7.78
BitFit	0.120%	69.81	1,407	12.7	7.33
Adapter	1.387%	71.54	1,509	12.8	9.74
AdaptFormer	1.387%	71.17	1,436	12.8	10.23
LoRA, all linear	1.375%	**71.98**	888	19.9	7.61

Where to train. Small changes in every block give much more accuracy per parameter than fine-tuning the top blocks. BitFit trains 0.12% of the backbone and beats fine-tuning the last block (8.3%) by 1.8 points; LoRA (1.375%) matches the last four blocks (33%), and the Adapter (1.387%) beats the last two (16.5%). Each extra top block helps less (the first adds 6.0 points over the linear probe, the second 3.0, each further pair about 1), likely because frozen lower blocks cannot change their features for the new classes, while small changes in every block reach all of them. Among the efficient methods, LoRA is the most accurate (71.98%), followed by the Adapter (71.54%), AdaptFormer (71.17%) and BitFit (69.81%). But LoRA is only 0.44 points ahead of the Adapter, close to our noise level; its clear advantage is that it can be merged. The serial Adapter and the parallel AdaptFormer differ by 0.37 points, and they also use different scales (1.0 *vs*. 0.1), so we cannot say which placement is better. None reaches full fine-tuning (73.38%), although the project description expects parameter-efficient methods to do better on small data; one possible reason is that each class has few images, but the whole training set has 450,000, enough to fine-tune the full backbone.

Training memory. Training memory depends on how far back the gradients have to go, not on how many parameters are trained (Fig. 5, right). Partial fine-tuning adds about 1.5 GB per trained block, which extrapolates to 18.8 GB for all 12 blocks, the same as full fine-tuning (18.9 GB). BitFit and the two adapters train at most 1.4% of the backbone but need 12.7–12.8 GB, more than fine-tuning half the network ($K = 6$, 10.0 GB), because their gradients must reach the first block. They still use a third less memory than full fine-tuning and train 24–32% faster, since frozen weight matrices need no weight gradients; LoRA is the exception (Section 4.6). At inference, only the two adapters cost extra: 9.74 and 10.23 ms at batch size 1, against 7.33 ms for the plain model.

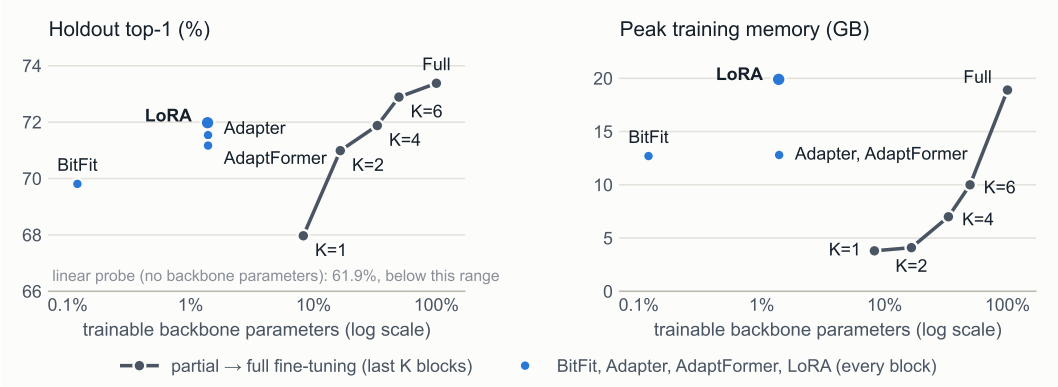


Figure 5: **Holdout accuracy (left) and peak training memory (right) against the share of trainable backbone parameters** (log scale) for the runs in Table 5. Methods that change every block (blue) get more accuracy per parameter than fine-tuning the last K blocks (gray), but need more memory, because their gradients go back to the first block. The linear probe (61.9%) is below the left panel; the memory of $K = 1$ is the feature-extraction peak.

Table 6: **Input resolution** (after the deadline; full fine-tuning, prototype head, 3 epochs, holdout top-1, %; runs `1019_1120_1_full` and `1105_1031_1_full_288`). Position embeddings are interpolated to the patch grid of each size. Bold: best test size for each model.

	Test size (pixels)							
Trained at	224	256	288	320	352	384	GPU-h	GB
224 pixels	73.38	75.21	**75.50**	75.28	74.33	72.94	0.48	18.9
288 pixels	73.10	75.74	77.23	**77.78**	77.63	77.41	0.68	30.2

4.6 LoRA: where to add capacity, and how long to train

Tables 9 and 10 in the appendix give the numbers. **Capacity mattered more than the learning rate.** With the LP head, halving the LoRA learning rate changed nothing (-0.12), while adding LoRA to all four linear layers instead of only `qkv` added 0.88 points. **LoRA is not cheaper to train here.** LoRA on all linear layers trains slower than full fine-tuning (885 *vs*. 1,131 img/s) and needs slightly more memory (19.9 *vs*. 18.9 GB). To train its A matrices, it must store the input of every linear layer, as full fine-tuning does, plus the activations of its 96 extra small matrix products; its savings are in stored size (35.5 MB *vs*. 374 MB per checkpoint). **Longer training helps full fine-tuning but not LoRA.** Going from 3 to 8 epochs adds 2.21 points for the final full fine-tuning recipe. LoRA stops improving after about 3 epochs, and with the prototype head 6 epochs are worse than 3 (-0.46), although its training loss keeps falling (1.95 to 1.70). We think the head is the cause: it holds 7.69M of LoRA's 8.87M trainable parameters, and compared with full fine-tuning our LoRA recipe trains it with a 10× higher learning rate, a 500× smaller weight decay and no drop path, so with more epochs it mostly memorizes the 43 images per class (not yet tested).

4.7 After the deadline: input resolution

After the deadline, the gap to the top teams on the leaderboard made us ask what still limits our models, so we changed only the image size, with the backbone and protocol of Table 5. timm interpolates the 14×14 position embeddings to the patch grid of the input (18×18 at 288 pixels), so no parameter is added (Table 6). Testing our 224-pixel model at 288 pixels adds 2.1 points with no retraining. FixRes (Touvron et al., 2019) explains this: random-resized crops make objects look larger during training, so slightly larger test images match them better. Training at 288 pixels adds another 2.3 points, and the best test size moves up to 320 pixels: 77.78%, 4.4 points above the 224-pixel model. This 3-epoch run still fits the budget (0.68 GPU-hours), but needs 30.2 GB instead of 18.9 GB, because a 288-pixel image has 325 tokens instead of 197. We also trained the full 8-epoch recipe at

288 pixels on all images; it takes 1.61 GPU-hours, over the budget, and has no test score, so it is not one of our results. Input resolution gave the largest gain we found, but we found it too late.

5 Conclusion

When adapting ViT-B/16 to 10,000 classes with 45 images each, the biggest gain came from how we started the new head, not from the adaptation method: class prototypes instead of a linear probe added 2.1–3.1 points, and gave 76.09% (full fine-tuning) and 71.67% (LoRA, 1.375% of the backbone) on the test set. Across seven methods, small changes in every block gave more accuracy per parameter than fine-tuning whole top blocks, with 12–69$\times$ fewer parameters, and training memory followed how far back the gradients go.

What we learned. First, failed ideas helped: halving LoRA's learning rate changed nothing, which pointed us from the optimizer to the head, and training LoRA longer lowered its training loss but also its accuracy, which showed that memorization, not underfitting, limits it. Second, a good start can be misleading: the linear-probe head starts 18 points ahead and ends 2.5 points behind, so we judged every choice on held-out data after training, never on the training loss or the starting accuracy. Third, we studied the adaptation method early and the input late: after the deadline, larger input images added 4.4 holdout points (Section 4.7), more than any single adaptation choice.

Limitations and next steps. Each setting was trained once (seed 0), and the holdout noise is about ± 0.3 points, so small differences, such as the 0.26-point gain from two extra images per class, are uncertain. Our explanation of why the linear-probe head hurts fits the loss curves and the random-head control, but it is not a full proof. The efficient methods share one learning rate that we did not tune per method, we only used LoRA rank 8, and some pairs (partial $K = 1, 2$ *vs.* $4, 6$; Adapter *vs.* AdaptFormer) differ in more than one setting. Next, we would trade epochs for image size within the budget (*e.g.* 6 epochs at 224 pixels and 2 at 288, about 1.2 GPU-hours by our estimate), test a lower head learning rate for LoRA, and try prompt-based methods.

References

Hangbo Bao, Li Dong, Songhao Piao, and Furu Wei. BEiT: BERT pre-training of image transformers. In *International Conference on Learning Representations (ICLR)*, 2022.

Elad Ben Zaken, Yoav Goldberg, and Shauli Ravfogel. BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.

Shoufa Chen, Chongjian Ge, Zhan Tong, Jiangliu Wang, Yibing Song, Jue Wang, and Ping Luo. AdaptFormer: Adapting vision transformers for scalable visual recognition. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009.

Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for NLP. In *International Conference on Machine Learning (ICML)*, 2019.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

Gao Huang, Yu Sun, Zhuang Liu, Daniel Sedra, and Kilian Q. Weinberger. Deep networks with stochastic depth. In *European Conference on Computer Vision (ECCV)*, 2016.

Menglin Jia, Luming Tang, Bor-Chun Chen, Claire Cardie, Serge Belongie, Bharath Hariharan, and Ser-Nam Lim. Visual prompt tuning. In *European Conference on Computer Vision (ECCV)*, 2022.

Simon Kornblith, Jonathon Shlens, and Quoc V. Le. Do better ImageNet models transfer better? In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2012.

Ananya Kumar, Aditi Raghunathan, Robbie Jones, Tengyu Ma, and Percy Liang. Fine-tuning can distort pretrained features and underperform out-of-distribution. In *International Conference on Learning Representations (ICLR)*, 2022.

Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2021.

Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations (ICLR)*, 2017.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, and Benjamin Bossan. PEFT: State-of-the-art parameter-efficient fine-tuning methods. https://github.com/huggingface/peft, 2022.

Hang Qi, Matthew Brown, and David G. Lowe. Low-shot learning with imprinted weights. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

Jake Snell, Kevin Swersky, and Richard Zemel. Prototypical networks for few-shot learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

Andreas Steiner, Alexander Kolesnikov, Xiaohua Zhai, Ross Wightman, Jakob Uszkoreit, and Lucas Beyer. How to train your ViT? Data, augmentation, and regularization in vision transformers. *Transactions on Machine Learning Research (TMLR)*, 2022.

Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

Hugo Touvron, Andrea Vedaldi, Matthijs Douze, and Hervé Jégou. Fixing the train-test resolution discrepancy. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

Ross Wightman. PyTorch image models. https://github.com/huggingface/pytorch-image-models, 2019.

Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. How transferable are features in deep neural networks? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.

A Training-log excerpts

The project asks us to back up training time and VRAM with training logs. Below are parts of `train_log.txt` for our two final models; the full logs (every 100 steps) are available on request. Dates are removed from the timestamps, Google Drive paths are shortened, and long lines are wrapped. GPU-hours include head initialization (feature extraction), training and the holdout check after every epoch (Section 3.1).

Final full fine-tuning model (prototype head, all 45 images per class, 8 epochs, flip TTA at prediction):

```
========== 1019_1319_1_full ==========
[13:32:14] full: lr 0.0001, head lr 0.0001, wd 0.05, layer decay 0.75, epochs 8,
           holdout 0/class, head init proto, time budget 1.20 h
[13:32:14] Environment: NVIDIA A100-SXM4-80GB (80 GB), bfloat16, 12 CPUs |
           torch 2.11.0+cu130 (CUDA 13.0), timm 1.0.30, peft 0.21.1
[13:38:32] Extracted features for 450,000 train / 0 holdout images in 6.1 min
[13:38:32] Head initialized from proto (6.2 min incl. feature extraction,
           counted in this run's GPU time)
[13:38:32] Trainable (excl. head): 85,798,656 = 100.000% of backbone |
           head 7,690,000 params (fine-tuned)
[13:38:32] Training up to 8 epochs x 1757 steps (batch 256x1, warmup 1757 steps)
[13:40:03] Speed 1146 img/s, 6.5 min/epoch
[13:46:13] == epoch 1: train loss 3.4158, 13.8 min used
[13:52:55] == epoch 2: train loss 2.6899, 20.5 min used
[13:59:51] == epoch 3: train loss 2.3729, 27.4 min used
[14:06:42] == epoch 4: train loss 2.1758, 34.2 min used
[14:13:40] == epoch 5: train loss 2.0321, 41.1 min used
[14:20:31] == epoch 6: train loss 1.9367, 47.9 min used
[14:27:32] == epoch 7: train loss 1.8741, 54.9 min used
[14:34:26] == epoch 8: train loss 1.8491, 61.8 min used
[14:34:29] Training done: 14056 steps, 1.03 GPU-hours (budget 1.20 h)
[14:34:29] Peak VRAM: 18.9 GB allocated, 20.1 GB reserved by PyTorch (GPU has 80 GB)
[14:34:32] Inference latency (batch 1): 14.95 ms/img | throughput (batch 256): 1919 img/s |
           NVIDIA A100-SXM4-80GB, bfloat16, with flip TTA
[14:35:29] [val] 50,000 predictions over 9,992 classes (max 13 per class) ->
           .../submission_val_1019_1319_1_full.zip
```

Final LoRA model (LoRA on all linear layers, prototype head, 3 epochs, 2 images per class held out, no TTA):

```
========== 1019_1319_3_lora ==========
[14:57:51] lora: lr 0.001, head lr 0.001, wd 0.0001, layer decay 1.0, epochs 3,
           holdout 2/class, head init proto, time budget 1.20 h
[14:57:51] Environment: NVIDIA A100-SXM4-80GB (80 GB), bfloat16, 12 CPUs |
           torch 2.11.0+cu130 (CUDA 13.0), timm 1.0.30, peft 0.21.1
[15:06:07] Extracted features for 430,000 train / 20,000 holdout images in 7.2 min
[15:06:08] Prototype head (zero-shot, 0 trainable params): holdout top-1 44.01%
[15:06:08] Head initialized from proto (7.2 min incl. feature extraction,
           counted in this run's GPU time)
[15:07:15] Trainable (excl. head): 1,179,648 = 1.375% of backbone |
           head 7,690,000 params (fine-tuned)
[15:07:15] Training up to 3 epochs x 1679 steps (batch 256x1, warmup 503 steps)
[15:07:53] Speed 888 img/s, 8.1 min/epoch
[15:15:44] == epoch 1: train loss 3.2174, holdout top-1 62.94%, 15.7 min used
[15:24:12] == epoch 2: train loss 2.4125, holdout top-1 70.12%, 24.2 min used
[15:32:40] == epoch 3: train loss 1.9496, holdout top-1 71.97%, 32.6 min used
[15:32:41] Training done: 5037 steps, 0.54 GPU-hours (budget 1.20 h)
[15:32:41] Peak VRAM: 19.9 GB allocated, 21.3 GB reserved by PyTorch (GPU has 80 GB)
[15:33:00] Final holdout: top-1 71.98%, top-5 89.07%
[15:33:03] Inference latency (batch 1): 7.61 ms/img | throughput (batch 256): 3869 img/s |
           NVIDIA A100-SXM4-80GB, bfloat16, no flip TTA
[15:33:55] [val] 50,000 predictions over 9,991 classes (max 15 per class) ->
           .../submission_val_1019_1319_3_lora.zip
```

B All runs

Table 7: **All 26 runs.** IDs encode the batch start time (month, day, hour, minute) and the position in the batch; the last block is the method comparison (Section 4.5). LoRA all: LoRA on all linear layers. Trainable: backbone parameters without the 7.69M head. GB: peak allocated training memory ($K = 1$: feature extraction). ms: batch-1 latency. ‡Flip TTA. *RTX PRO 6000 Blackwell; all other runs on an A100-SXM4-80GB (speed and latency are not comparable). The LP head (62.07%) is computed inside every LP run; `1102_1149_2` re-ran it alone.

Run	Method	Head	Ep.	Trainable	Holdout	Val	GPU-h	GB	ms
`0928_0636_1_linear`	Zero-shot	prototype	0	0	43.95	–	0.11	3.9	7.55
`0928_0636_3_full`	Full FT	LP	3	100%	70.88	–	0.45	18.9	7.31
`0928_0748_1_full`	Full FT	LP	8	100%	72.31	72.57/72.84‡	1.03	18.9	7.35
`1018_2306_1_full`	Full FT, all data	LP	8	100%	–	73.10‡	1.03	18.9	14.92‡
`1019_1120_1_full`	Full FT	prototype	3	100%	73.38	–	0.48	18.9	7.78
`1019_1319_2_full`	Full FT, all data	prototype	3	100%	–	74.01‡	0.44	18.9	15.24‡
`1019_1319_1_full`	**Full FT, all data (final)**	prototype	8	100%	–	**76.22**‡	1.03	18.9	14.95‡
`0928_0636_2_lora`	LoRA qkv	LP	3	0.344%	68.57	–	0.40	13.5	7.22
`0928_0748_2_lora`	LoRA qkv, head lr 2×10^{-4}	LP	10	0.344%	68.69	–	1.06	13.5	7.53
`1018_1239_1_lora`	LoRA qkv, lr 5×10^{-4}	LP	3	0.344%	68.45	–	0.41	13.5	7.15
`1018_1239_3_adaptformer`	AdaptFormer	LP	3	1.387%	69.05	–	0.39	12.8	9.89
`1018_1239_2_lora`	LoRA all	LP	3	1.375%	69.45	–	0.55	19.9	7.03
`1018_2306_2_lora`	LoRA all	LP	6	1.375%	69.87	–	0.97	19.9	7.31
`1019_0953_1_lora`*	LoRA all	random	3	1.375%	71.95	–	0.37*	19.9	1.81*
`1019_0953_2_lora`*	LoRA all	prototype	3	1.375%	72.00	–	0.44*	19.9	1.82*
`1019_1120_2_lora`	LoRA all	prototype	6	1.375%	71.52	–	0.97	19.9	7.76
`1019_1319_3_lora`	**LoRA all (final)**	prototype	3	1.375%	**71.98**	**71.90**	0.54	19.9	7.61
`1102_1149_1_linear_proto`	Zero-shot (re-run)	prototype	0	0	43.95	–	0.10	3.8	7.32
`1102_1149_2_linear_lp`	Linear probe	LP	–	0	61.94	–	0.10	3.2	6.96
`1102_1942_1_partial_k1`	Partial FT, $K = 1$	prototype	3	8.263%	67.97	–	0.35	3.8	7.92
`1102_1942_2_partial_k2`	Partial FT, $K = 2$	prototype	3	16.524%	70.99	–	0.35	4.1	7.41
`1102_1942_3_partial_k4`	Partial FT, $K = 4$	prototype	3	33.046%	71.88	–	0.35	7.0	7.41
`1102_1942_4_partial_k6`	Partial FT, $K = 6$	prototype	3	49.568%	72.89	–	0.35	10.0	7.42
`1102_2023_1_bitfit`	BitFit	prototype	3	0.120%	69.81	–	0.39	12.7	7.33
`1102_2017_1_adapter`	Adapter	prototype	3	1.387%	71.54	–	0.37	12.8	9.74
`1102_2212_1_adaptformer`	AdaptFormer	prototype	3	1.387%	71.17	–	0.39	12.8	10.23

Runs after the deadline (Section 4.7). None of them is used for a ranked result. Batch `1105_0141` ran zero-shot and the linear probe with the ImageNet-21k-only checkpoints `augreg_in21k` and `orig_in21k` (zero-shot 49.96% and 42.43%, linear probe 61.05% and 51.07%; Section 3.1), and evaluated `1019_1120_1_full` at six test sizes. `1105_1031_1_full_288` trains 3 epochs at 288 pixels on the holdout split (0.68 GPU-h, 30.2 GB). `1105_1032_1_full_288` trains the final recipe at 288 pixels on all images (1.61 GPU-h, 30.2 GB); it was not submitted, so it has no test score.

C Hardware, software and the path to the final model

All runs use one NVIDIA A100-SXM4-80GB with 12 CPU cores, except one ablation batch (random and prototype heads for LoRA in Table 4), which ran on an RTX PRO 6000 Blackwell. Repeating its prototype run on the A100 gave 71.98% *vs*. 72.00%, so accuracies are comparable across the two GPUs; speeds and latencies are not. Software, as logged from batch `1019_1120` on, which covers all final models and the method comparison: PyTorch 2.11.0 (CUDA 13.0), timm 1.0.30 and PEFT 0.21.1. Earlier runs used the then-current releases, which were not logged. Peak memory is the memory allocated by PyTorch; the memory it reserved is 0.2–1.4 GB higher (20.1 GB for the final model). Figure 6 shows the validation accuracy of the final full fine-tuning recipe as we changed one setting at a time.

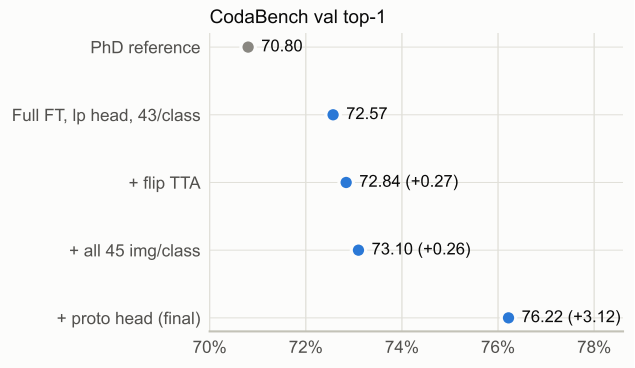


Figure 6: **CodaBench validation accuracy as we change one setting at a time** (all runs: full fine-tuning, 8 epochs). The gray dot is the reference. Positions encode values, so the axis starts at 70%.

D Extra tables

Table 8 shows how the trainable parameters of each method are counted (Section 3.3). Tables 9 and 10 give the numbers behind Section 4.6.

Table 8: **Trainable backbone parameters** (head excluded; $r = 8$; partial: $K \in \{1, 2, 4, 6\}$). 1,536: final LayerNorm (partial) or the patch-embedding and final-norm biases (BitFit); 8,448 biases per block. Adapter: $2 \cdot 768 \cdot 64 + 64 + 768 = 99{,}136$ per block. LoRA adds $r\,(d_{\text{in}} + d_{\text{out}})$ per layer, *i.e.* $3{,}072\,r$ for qkv and $12{,}288\,r$ for qkv, proj, fc1 and fc2 together.

Method	Count	Parameters	%
Partial, last K blocks	$7{,}087{,}872\,K + 1{,}536$	7.09M–42.5M	8.26–49.6
Full fine-tuning	all	85,798,656	100
BitFit	$12 \cdot 8{,}448 + 1{,}536$	102,912	0.120
Adapter, AdaptFormer	$12 \cdot 99{,}136$	1,189,632	1.387
LoRA, qkv only	$12 \cdot 3{,}072\,r$	294,912	0.344
LoRA, all linear	$12 \cdot 12{,}288\,r$	1,179,648	1.375

Table 9: **PEFT design** (LP head, 3 epochs, holdout top-1, %). Params: trainable backbone parameters (head excluded). img/s: training throughput; GB: peak training memory; ms: batch-1 latency after merging (A100, bf16).

Method	Params	Holdout	img/s	GB	ms
LoRA qkv, lr 10^{-3}	0.344%	68.57	1,341	13.5	7.22
LoRA qkv, lr 5×10^{-4}	0.344%	68.45	1,353	13.5	7.15
LoRA all linear	1.375%	**69.45**	885	19.9	7.03
AdaptFormer	1.387%	69.05	1,437	12.8	9.89
Full fine-tuning	100%	70.88	1,131	18.9	7.31

Table 10: **Training length** (holdout top-1, %; LoRA all: LoRA on all linear layers). ‡All 45 images per class with flip TTA, CodaBench validation. †The 10-epoch run also lowered the head learning rate to 2×10^{-4}.

Setting	Short (ep)	Long (ep)	Δ
Full FT, LP	70.88 (3)	72.31 (8)	+1.43
Full FT, prototype, final‡	74.01 (3)	76.22 (8)	+2.21
LoRA qkv, LP†	68.57 (3)	68.69 (10)	+0.12
LoRA all, LP	69.45 (3)	69.87 (6)	+0.42
LoRA all, prototype	71.98 (3)	71.52 (6)	−0.46

E HYPERPARAMETERS

Table 11: **Hyperparameters.** Weight decay is not applied to biases, normalization parameters, the class token and position embeddings. Evaluation resizes to 248 pixels and takes a 224 center crop.

	Full fine-tuning	LoRA	AdaptFormer	Linear-probe head
Trainable	all weights + head	A, B in qkv, proj, fc1, fc2 (all 12 blocks) + head	parallel MLP adapters + head	head only, on cached features
Method-specific	layer-wise lr decay 0.75, drop path 0.1	$r = 8$, $\alpha = 16$, no dropout, no layer decay, no drop path, merged after training	bottleneck 64, scale 0.1, dropout 0.1	starts from the prototype head
Optimizer	AdamW	AdamW	AdamW	AdamW
Learning rate	10^{-4} (head 10^{-4})	10^{-3} (head 10^{-3})	10^{-3} (head 10^{-3})	10^{-3}
Weight decay	0.05	10^{-4}	10^{-4}	10^{-4}
Schedule	linear warmup (1 epoch; 0.3 epochs for 3-epoch runs), then cosine decay to 0			cosine, no warmup
Epochs	3 or 8	3, 6 or 10	3	10
Batch size	256	256	256	4,096
Precision	bf16	bf16	bf16	fp32 head, fp16 features
Regularization	label smoothing 0.1, gradient clipping at norm 1.0			label smoothing 0.1
Augmentation	RandomResizedCrop (scale 0.25–1) + horizontal flip			none (center crops)

Methods added for Section 4.5 train for 3 epochs with the settings above, except: partial fine-tuning uses learning rate 3×10^{-4} and weight decay 0.05 for $K = 1, 2$ and the full fine-tuning settings for $K = 4, 6$ (drop path only in trained blocks); BitFit uses learning rate 10^{-3} and no weight decay; the Adapter is AdaptFormer's bottleneck placed after the MLP, with scale 1.0.